PROJECT REPORT

FOR

Foundations of Artificial Intelligence (CS23533)

AI-BASED SMART BANK QUEUE BALANCER SYSTEM

A PROJECT REPORT

Submitted by

2116240701189 – HEMALATHA ST

2116240701190 – HEMASHREE R



November, 2026

BONAFIDE CERTIFICATE

Certified that this project report "**AI-BASED SMART BANK QUEUE BALANCER SYSTEM**" is the bonafide work of "**2116240701189 – HEMALATHA ST & 2116240701190 – HEMASHREE R**" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
	ABSTRACT	**1**
1	**INTRODUCTION**	**2**
	1.1 BACKGROUND AND MOTIVATION	2
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE AND ORGANISATION OF THE REPORT	3
2	**LITERATURE REVIEW**	**4**
	2.1 QUEUEING THEORY AND QUEUE DISCIPLINES	4
	2.2 AI TECHNIQUES FOR DECISION MAKING	4
	2.3 EXISTING SYSTEMS AND RESEARCH GAP	5
3	**PROPOSED SYSTEM**	**6**
	3.1 ARCHITECTURE DIAGRAM	6
	3.2 AI DECISION PIPELINE	7
	3.3 DATA MODEL AND TICKET LIFE CYCLE	8
	3.4 TECHNOLOGY STACK	9
4	**MODULES DESCRIPTION**	**10**
	4.1 INTELLIGENT AGENT AND RECOMMENDER MODULE	10
	4.2 HEURISTIC AND GREEDY BEST-FIRST MODULE	11
	4.3 CONSTRAINT SATISFACTION MODULE	11
	4.4 BACKTRACKING MODULE	12
	4.5 FORWARD CHAINING MODULE	12
	4.6 BACKWARD CHAINING RULE ENGINE	13
	4.7 ATOMIC QUEUE AND TOKEN ENGINE	13
	4.8 REAL-TIME SYNCHRONISATION MODULE	14
	4.9 SECURITY AND ACCESS CONTROL MODULE	14
	4.10 CUSTOMER, STAFF AND ADMIN PORTALS	14
5	**IMPLEMENTATION AND RESULTS**	**16**
	5.1 EXPERIMENTAL SETUP	16
	5.2 RESULTS	17
	5.3 OUTPUT	23
6	**CONCLUSION AND FUTURE WORK**	**25**
7	**REFERENCES**	**26**

ABSTRACT

Long and unevenly distributed waiting lines are a persistent problem in bank branches, where customers choose between parallel counters with no information about their real load. This project presents an **AI-Based Smart Bank Queue Balancer System** that replaces guesswork with rational, explainable decisions. A permanent QR poster at the branch opens a mobile web application in which customers select a service and receive a recommended queue, a token and a live ticket.

The decision logic is a modular AI engine built from syllabus techniques: a rational-agent recommender, the heuristic h(q) = waitingCount × averageServiceTime, greedy best-first selection, a constraint satisfaction problem with four constraints, backtracking as a fallback, forward chaining for queue-state inference and backward chaining to prove eligibility to join. Every recommendation is returned with a reasoning trace. Atomic database updates, strict FIFO ordering, role-based access control and Socket.IO broadcasts keep all customer, staff and admin screens consistent.

The system was implemented with React, Node.js, Express, MongoDB and Socket.IO. In a simulation using the project's own recommendation function, AI balancing reduced mean queue imbalance by 52–81% and the 95th-percentile wait by 14–34% against random queue choice, and cut turn-aways under heavy load from 4.7% to 1.6%.

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Banks remain one of the most visited service institutions, and a typical branch offers several services such as cashier, deposit, account opening and loan enquiry, each served by more than one counter. When customers arrive they usually join whichever line looks shortest at a glance. They cannot see how long each customer ahead will take, whether another counter has just opened, or whether a line is about to overflow. The result is a familiar pattern: one counter is overloaded while its neighbour is nearly idle, average waiting time rises and customers lose confidence in the process.

Queue behaviour is a well-studied area of operations research (Kleinrock, 1975; Gross and Harris, 1998), and the economic and psychological cost of waiting has been documented extensively (Larson, 1987). Artificial Intelligence offers a complementary view: the choice of a queue is a decision made by an agent that perceives its environment, evaluates alternatives against an objective and acts to maximise a measure of performance (Russell and Norvig, 2021). This project applies that view to a real banking scenario and turns the principles studied in the course Foundations of Artificial Intelligence into a working, deployable application.

1.2 Problem Statement

Existing branch queues suffer from four related problems. First, load is unbalanced across parallel counters of the same service. Second, customers have no reliable estimate of waiting time. Third, manual token systems and paper slips do not provide live status or protect against overflow. Fourth, concurrent arrivals can corrupt shared counters if the software is not designed carefully. The problem addressed here is to design a system that, for every arriving customer, selects a valid and optimal queue, explains why, and keeps the shared queue state correct under concurrent access in real time.

1.3 Objectives

- To provide a permanent, never-changing branch QR code that opens the digital queue for any customer without installing an application.
- To recommend the best queue for each customer using heuristic search, constraint satisfaction and logical inference, and to present a human-readable reasoning trace.

- To guarantee correctness of queue counts, strict first-in-first-out service and a single active ticket per customer, even under simultaneous requests.
- To synchronise customer, staff and administrator screens in real time using WebSockets, including a call alert (chime and vibration) when a customer's turn arrives.
- To give the branch administrator control of services, queues, staff and end-of-day reporting, and to secure every role through authentication and role-based access control.
- To map each AI technique in the syllabus (Units I–IV) to a clearly identifiable module that is invoked during live decisions.

1.4 Scope and Organisation of the Report

The scope of the project is a single bank branch with multiple services, each with parallel queues of fixed capacity. The system covers the customer, staff and admin workflows end to end. Payment, core-banking integration and biometric identification are outside the scope.

The remainder of the report is organised as follows. Chapter 2 reviews related work. Chapter 3 presents the proposed system, its architecture and its data model. Chapter 4 describes each module in detail. Chapter 5 explains the experimental setup and reports results. Chapter 6 concludes and lists future work, and Chapter 7 lists the references.

CHAPTER 2

LITERATURE REVIEW

2.1 Queueing Theory and Queue Disciplines

Queueing theory models customers arriving at a service facility, waiting for their turn and being served. Little (1961) proved the relation $L = \lambda W$ between the average number of customers in a system, their arrival rate and the average time spent, a result that holds under very general conditions. For a single line, the expected delay seen by a newly arrived customer is therefore proportional to the number of people already waiting and the mean service time. This relation is the theoretical basis for the heuristic used in this project.

For systems with several parallel servers, Winston (1977) showed that joining the shortest line is optimal in a broad class of situations when service times are exponential, and later work extended this result. Haight (1957) studied balking, where customers decline to join a line that appears too long, which corresponds to the capacity limit enforced here. Larson (1987) argued that fairness and the perception of waiting matter as much as its length, which motivates strict first-in-first-out ordering and transparent status information.

2.2 AI Techniques for Decision Making

Intelligent agents: An agent maps percepts to actions and is rational if it chooses the action expected to maximise its performance measure (Russell and Norvig, 2021). A queue recommender fits this definition directly: the percepts are queue states, the actions are queue choices and the performance measure is waiting time.

Heuristic search: Hart, Nilsson and Raphael (1968) formalised the use of heuristic functions to guide search, and Pearl (1984) gave a detailed account of heuristic strategies including greedy best-first search, which always expands the option with the lowest heuristic estimate.

Constraint satisfaction: A CSP consists of variables, domains and constraints, and solutions are found by pruning inconsistent values (Mackworth, 1977; Dechter, 2003). Backtracking, described by Golomb and Baumert (1965), systematically retracts a failed choice and tries the next alternative.

Logical inference: Knowledge-based systems derive new facts from known facts using forward chaining (data-driven) and prove goals using backward chaining (goal-driven) (Brachman and Levesque, 2004; Russell and Norvig, 2021).

2.3 Existing Systems and Research Gap

Conventional token dispensers and generic queue-management products order customers but usually make no attempt to reason about which counter is best for a particular customer. Appointment systems reduce walk-ins but do not help the large share of customers who arrive unannounced. Table 2.1 compares common approaches with the proposed system.

Approach	How customer is assigned	Limitation	Proposed system
Manual lines / paper token	Customer picks a counter by eye	No load information; frequent imbalance	AI picks the lowest-wait valid queue
Single shared FIFO line	Next free counter takes next customer	Needs floor space; no service-type routing	Per-service parallel queues with capacity limits
Appointment booking	Fixed time slot	Does not handle walk-ins	QR walk-in with live estimate
Basic digital token	Counter chosen by customer	No recommendation or explanation	Explainable recommendation with reasoning trace

Table 2.1 Comparison of queue management approaches

The gap addressed by this project is the combination of (i) an explainable AI decision layer built from several complementary techniques, (ii) rigorous concurrency control so that the decision remains valid at the moment it is executed, and (iii) a zero-install access path through a permanent QR code.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is a full-stack web application for a single bank branch. A customer scans the permanent branch QR poster, which resolves to the branch address `/bank/ABC001`. After a quick sign-in the customer sees every service with its live parallel queues, asks the AI engine for a recommendation, confirms a queue and receives a digital ticket. Staff call customers in strict order from their own desk, and the administrator monitors and configures the branch. Table 3.1 summarises the system as a rational agent using the PEAS description.

Component	Description
Performance measure	Minimise estimated waiting time of the arriving customer; keep parallel queues balanced; never exceed capacity; preserve FIFO fairness.
Environment	One bank branch; services with 1–10 parallel queues; customers arriving at random; staff completing services at random durations.
Actuators	Queue recommendation and reasoning trace; token issue; ticket status; real-time broadcasts and call alerts.
Sensors	Queue status (OPEN/CLOSED), waitingCount, capacity, average service time, customer's active-ticket status.

Table 3.1 PEAS description of the queue-balancing agent

3.1 Architecture Diagram

Figure 3.1 shows the layered architecture. The React front end talks to an Express REST API protected by JWT authentication, role-based access control and rate limiting. Requests that need a decision pass through the AI reasoning engine, which consists of seven cooperating modules. Requests that change state go to the atomic queue engine, which writes to MongoDB, the single source of truth, and then triggers Socket.IO broadcasts so that every connected screen refreshes without reloading.

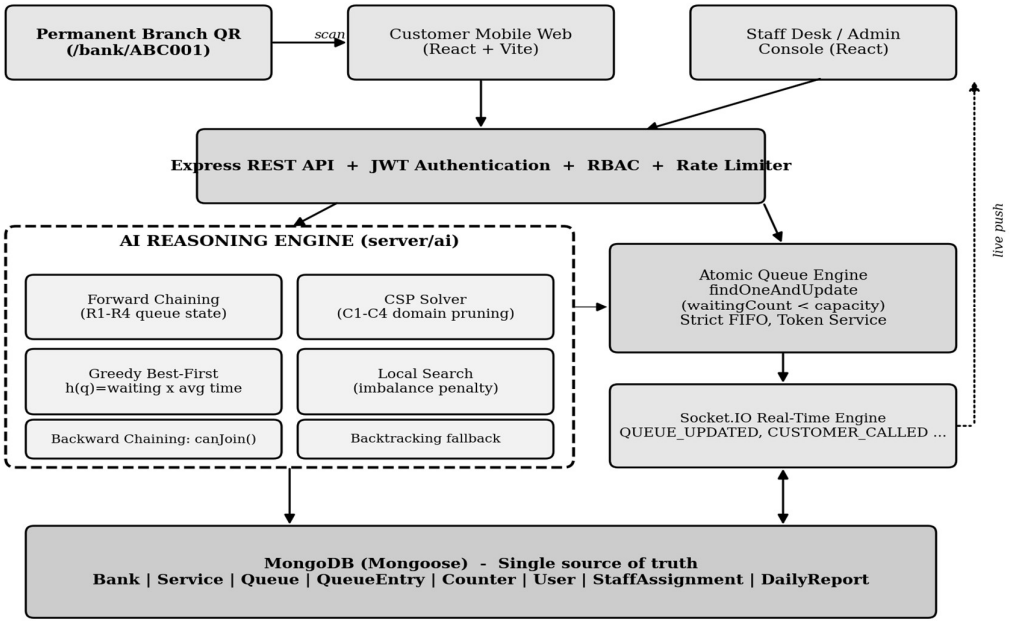


Figure 3.1 System architecture of the AI-Based Smart Bank Queue Balancer

3.2 AI Decision Pipeline

Each recommendation follows the pipeline in Figure 3.2. The same modules are reused at confirmation time, which is where a stale recommendation is detected and repaired.

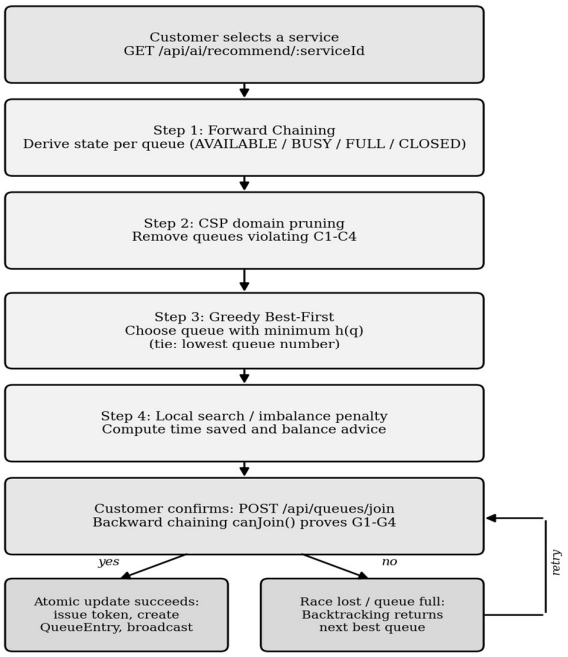


Figure 3.2 AI decision pipeline from recommendation to confirmed join

1. **Forward chaining** converts raw facts (status, waitingCount, capacity) into a derived state for each queue.
2. **CSP pruning** removes every queue that violates a constraint, leaving only consistent candidates.
3. **Greedy best-first selection** chooses the candidate with the smallest heuristic value; ties are broken deterministically by the lowest queue number.
4. **Local-search evaluation** measures queue imbalance, computes the time the customer saves and produces balancing advice.
5. **Backward chaining** proves the goal "customer C can join queue Q" at confirmation time; if it fails or the atomic update loses a race, **backtracking** selects the next best consistent queue.

3.3 Data Model and Ticket Life Cycle

Eight MongoDB collections hold the system state (Table 3.2). The field Queue.waitingCount is the authoritative count of customers currently in the WAITING state and is never allowed to become negative or to exceed capacity.

Collection	Purpose and key fields
Bank	Branch identity, bank code (e.g. ABC001) and the permanent QR URL.
Service	Service name and code, averageServiceTime, numberOfQueues, queueCapacity, ACTIVE/INACTIVE status.
Queue	serviceId, queueNumber, capacity, OPEN/CLOSED status and waitingCount (authoritative).
QueueEntry	Customer ticket: token, sequence, status, timestamps (joined, called, completed), actual service duration, serving staff, archive flag.
Counter	Per-queue, per-day atomic sequence used to generate token numbers.
User	Customers, staff and administrators with hashed passwords and role.
StaffAssignment	Binds each staff member to exactly one service queue.
DailyReport	Summary created by the End Day process (counts, averages) for history.

Table 3.2 Collections and their purpose

A ticket moves through the life cycle shown in Figure 3.3. The waiting count is decremented when staff call the customer or when the customer cancels, never twice for the same ticket.

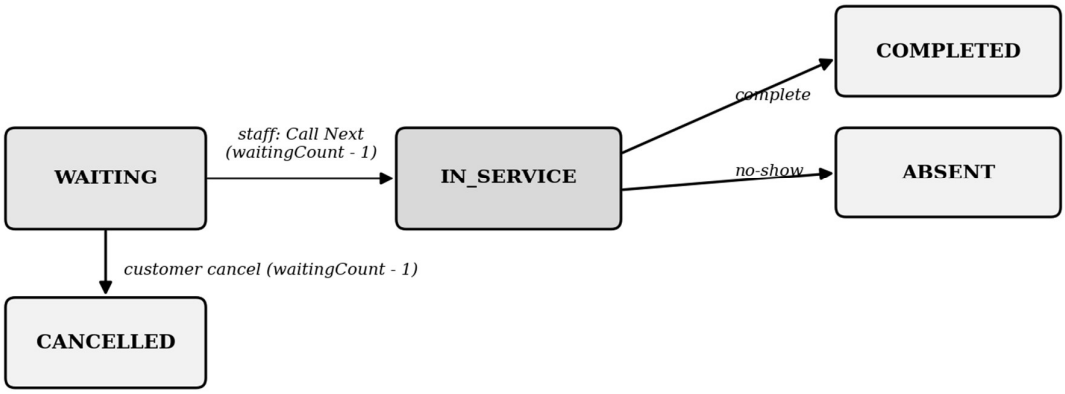


Figure 3.3 Life cycle of a queue entry (ticket)

3.4 Technology Stack

Layer	Technology	Role
Front end	React 18, Vite, React Router, Framer Motion, Recharts	Responsive customer, staff and admin interfaces; charts on the admin dashboard
QR / scanning	qrcode.react, html5-qrcode	Generate the permanent branch QR; scan it in-app
Back end	Node.js, Express 4, Helmet, CORS, express-rate-limit, express-validator	REST API and security middleware
Real time	Socket.IO 4	Live queue snapshots and call events
Database	MongoDB with Mongoose 8 (embedded mongodb-memory-server fallback)	Persistent data store with atomic updates
Security	JSON Web Tokens, bcryptjs	Stateless sessions and password hashing
AI engine	Plain JavaScript modules (server/ai)	Heuristic, CSP, logic and recommendation

Table 3.3 Technologies used

CHAPTER 4

MODULES DESCRIPTION

Every algorithm in the syllabus is implemented in its own file and is called during live queue decisions. Table 4.1 maps each concept to its module.

Unit	Concept	File / function	Role in the system
I	Intelligent agent, rationality	queueRecommendation.js recommendBestQueue()	Perceives queue state and picks the utility-maximising action
II	Heuristic function	queueHeuristic.js calculateQueueHeuristic()	h(q) = waitingCount × avgServiceTime
II	Greedy best-first	greedyBestFirst.js selectGreedyBestQueue()	Selects minimum-h queue with deterministic tie-break
II	Local search / optimisation	queueRecommendation.js balanceSuggestion	Imbalance measure and balancing advice
III	Constraint satisfaction	csp.js solveQueueCSP()	Prunes queues violating C1–C4
III	Backtracking	backtracking.js backtrackToAlternativeQueue()	Re-selects when preferred queue becomes invalid
IV	Propositional logic, forward chaining	forwardChaining.js inferQueueState()	Derives AVAILABLE / BUSY / FULL / CLOSED
IV	Backward chaining	ruleEngine.js canJoin()	Goal-directed proof of eligibility with trace

Table 4.1 Mapping of syllabus topics to modules

4.1 Intelligent Agent and Recommender Module

The function recommendBestQueue(queues, service, options) is the agent program. Its percept is the list of queues of the chosen service together with the service's average time and whether the customer already holds an active ticket. It runs the pipeline of Section 3.2 and returns the recommended queue, a per-queue table (derived state, heuristic value, estimated wait), the time saved in minutes, a balance suggestion and the complete reasoning and trace arrays. The utility the agent maximises is the reduction in waiting time:

timeSaved = max(0, max over other queues of estimatedWait − estimatedWait of recommended queue)

If the constraint-satisfaction domain is empty, the agent returns no recommendation and a clear message that all queues are full or closed. The customer interface displays the result in an expandable AI Insight panel so that the decision is transparent.

4.2 Heuristic and Greedy Best-First Module

The heuristic estimates how much work is queued in a line:

$$h(q) = \text{waitingCount}(q) \times \text{averageServiceTime}(\text{service}) \quad \text{(minutes)}$$

This follows directly from Little's relation (Little, 1961): the delay of a newly arriving customer equals the number of people ahead multiplied by the mean service time. A helper calculateEstimatedWaitTime(peopleAhead, avgServiceTime） provides the same estimate for a customer already holding a ticket.

selectGreedyBestQueue() evaluates h(q) for every consistent candidate and chooses the minimum, in the manner of greedy best-first search (Pearl, 1984). When two queues have the same value the module sorts them by queue number and selects the lowest, and it records an explanatory sentence such as "Queue 1 and Queue 2 both have an identical estimated waiting load of 4 mins; resolved by selecting Queue 1 (lowest queue index)". The deterministic rule makes the system reproducible and testable.

The local-search part of the recommender computes queue imbalance as the difference between the largest and smallest waitingCount. If the imbalance exceeds one customer, the system reports that traffic is being directed to the recommended queue to restore equilibrium.

4.3 Constraint Satisfaction Module

Queue assignment is formulated as a CSP. The variable is the customer's queue assignment X; its domain is the set of queues of the chosen service, D = {Q1, Q2, …, Qk}. The constraints are given in Table 4.2.

ID	Name	Condition	Effect when violated
C1	Capacity	waitingCount(Qi) < capacity(Qi)	Queue pruned as full
C2	Status	status(Qi) = OPEN	Queue pruned as closed
C3	Service match	serviceId(Qi) = target service	Queue pruned as mismatched
C4	Single active ticket	Customer has no WAITING / IN_SERVICE entry	Whole domain rejected

Table 4.2 Constraints of the queue-assignment CSP

solveQueueCSP() first checks the global constraint C4, then tests the unary constraints C1–C3 on each queue and returns the filtered domain, the violations for each pruned queue, a consistency flag and a step-by-step trace. Only queues that survive pruning are passed to the greedy selector, so the system can never recommend a full or closed queue.

4.4 Backtracking Module

A recommendation is computed at one instant but executed a moment later, and another customer may take the last free slot in between. backtrackToAlternativeQueue() handles this. It re-runs the CSP on fresh queue data, removes the invalid candidate from the domain, and applies greedy best-first selection to the remaining candidates. If an alternative exists, it returns that queue with a trace such as "Backtracked successfully to Queue 2 (6/10 waiting)". If none exists it reports failure and the reason. The join service attaches this result to the error response so the interface can immediately offer the next best queue.

4.5 Forward Chaining Module

inferQueueState() is a data-driven rule engine. The base facts are the queue status, waitingCount and capacity. Four rules fire in order and each records its derivation step:

- **R1:** IF status = CLOSED THEN state = CLOSED, canJoin = false.
- **R2:** IF status = OPEN AND waitingCount ≥ capacity THEN state = FULL, canJoin = false.
- **R3:** IF status = OPEN AND waitingCount < capacity AND waitingCount/capacity ≥ 0.6 THEN state = BUSY, canJoin = true.

- **R4:** IF status = OPEN AND waitingCount < capacity AND waitingCount/capacity < 0.6 THEN state = AVAILABLE, canJoin = true.

The derived state drives the colour-coded capacity gauges and queue cards in the interface.

4.6 Backward Chaining Rule Engine

canJoin(customer, queue, service, context) is a goal-directed prover. The top goal G0 "customer C can join queue Q for service S" is reduced to four sub-goals that are tested in order: G1 the queue is OPEN; G2 capacity is available; G3 the queue belongs to the service; G4 the customer has no active ticket. The function returns the proof trace with a pass or fail mark for each sub-goal, the failed goal identifier and a user-friendly reason. The join service calls it before any database write, so invalid requests are rejected with an explanation of exactly which sub-goal failed.

4.7 Atomic Queue and Token Engine

The queue service guarantees correctness under concurrency. The key operations are:

- **Join:** after rule verification, a single findOneAndUpdate with the filter status = OPEN and waitingCount < capacity increments waitingCount. MongoDB applies updates to one document one at a time, so when two customers compete for the last place exactly one update succeeds. The other triggers backtracking. If token creation or ticket insertion fails, the increment is rolled back.
- **Token generation:** a Counter document per queue per day is incremented atomically to produce tokens of the form SERVICECODE-Qn-001, for example CASH-Q1-001.
- **Call next (strict FIFO):** staff atomically move the oldest WAITING entry (sorted by sequence and joined time) to IN_SERVICE and decrement waitingCount with a guard that prevents it from dropping below zero. A staff member cannot call another customer while one is in service.
- **Complete / absent / cancel:** completing records the actual duration and recomputes the service's average time as a rolling mean of the last ten completed tickets (when at least three exist). Cancelling decrements waitingCount once only.
- **Ticket status:** position and estimated wait are computed from the number of WAITING entries with a lower sequence number.

4.8 Real-Time Synchronisation Module

A Socket.IO server authenticates clients with the JWT, places each user in a personal room and lets clients join branch and service rooms. After every state change the server broadcasts events so all screens update without reloading (Table 4.3). The most important is QUEUE_UPDATED, which carries a complete snapshot of the service's queues, avoiding partial-update inconsistencies. When a customer is called, CUSTOMER_CALLED triggers an audio chime and device vibration on the customer's ticket.

Event	Meaning
CUSTOMER_JOINED	A customer joined a queue (token, queue number, new waiting count)
CUSTOMER_CALLED	Staff called the customer; ticket alerts with chime and vibration
SERVICE_STARTED	Customer moved to IN_SERVICE
SERVICE_COMPLETED	Staff completed the service (with duration in minutes)
CUSTOMER_CANCELLED	Customer cancelled the ticket
QUEUE_UPDATED	Full snapshot of all queues of a service
QUEUE_STATUS_CHANGED	Administrator opened or closed a queue
DAY_RESET	End Day executed; counters reset to zero

Table 4.3 Real-time events

4.9 Security and Access Control Module

Passwords are stored as bcrypt hashes and sessions are carried by JSON Web Tokens (Jones, Bradley and Sakimura, 2015). Middleware verifies the token and then enforces the role: customers may join, cancel and view their own ticket; staff may only act on the single queue assigned to them; administrators manage services, queues, staff, the QR code and reports. Unauthorised calls receive explicit messages such as "Unauthorized staff" or "Unauthorized admin". Login endpoints are limited to 100 requests per IP in 15 minutes, a general limiter caps other traffic, and Helmet adds standard HTTP security headers.

4.10 Customer, Staff and Admin Portals

Customer portal: After scanning the branch QR, the customer registers or signs in (email or mobile), browses the branch services, sees live capacity gauges, reads the AI Insight panel and

joins the recommended queue. The ticket page shows the token, position, people ahead, estimated wait and a cancel option, and alerts when the customer is called.

Staff portal: Each staff member signs in to a counter desk bound to one queue, sees the waiting list and the customer in service, and uses the keyboard shortcuts N (call next) and C (complete). A customer who does not appear can be marked absent.

Admin portal: The administrator has a live dashboard with charts and a load heat-map, and pages to manage services (name, code, average time, number of queues, capacity), open or close individual queues, view the customers waiting in a queue, register and reassign staff, display and print the permanent QR code, run the End Day reset and browse historical daily reports. The End Day process archives the day's tickets, writes a DailyReport, resets all counts to zero and leaves the QR code valid.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

Implementation environment: The system was developed as a monorepo with a Node.js back end (Express, Mongoose, Socket.IO) and a React front end built with Vite. MongoDB 7.0 runs either as an external server (when a connection string is configured) or as an embedded persistent instance through mongodb-memory-server, so that the project runs on a fresh machine without installing a database. A seed script creates one branch (Apex Commercial Bank, Metro Central Branch, code ABC001), one administrator, ten staff and three demo customers, together with the five services listed in Table 5.1, each with two parallel queues of capacity ten.

Service	Code	Avg. service time (min)	Queues × capacity
Cashier	CASH	4	2 × 10
Deposit	DEP	5	2 × 10
Doubt Clarification	DOUBT	7	2 × 10
Account Opening	ACC	10	2 × 10
Loan Enquiry	LOAN	15	2 × 10

Table 5.1 Seeded services

Evaluation methods: Three complementary methods were used.

(1) *Unit-level scenario runs*: the AI modules were executed on controlled queue states to verify the decision, the derived state and the trace.

(2) *Acceptance tests*: the project includes an automated suite (npm test) that exercises the REST API end to end.

(3) *Policy simulation*: a discrete-time simulation drives the project's unmodified recommendBestQueue() function and compares it with two baseline policies.

Simulation design: One service with two parallel queues, each served by one counter, capacity ten per queue and a mean service time of about 4 minutes (exponentially distributed and rounded to whole minutes, minimum one minute). Time advances in one-minute steps over an 8-hour day (480 minutes). In each minute one customer arrives with probability λ. Three load

levels were used, with λ = 0.25, 0.375 and 0.475 customers per minute, giving nominal utilisations of 0.50, 0.75 and 0.95. Each configuration was repeated with 30 independent seeded runs and the mean of the runs is reported. Waiting time is measured from joining a queue until being called. Three policies were compared:

- **Always Queue 1:** Every customer joins the first queue (a naive reference).
- **Random queue:** Every customer picks a queue at random, as when no information is available.
- **AI-balanced(proposed):**The customer joins the queue returned by recommendBestQueue(). A customer is turned away only when no queue satisfies the capacity constraint.

5.2 Results

5.2.1 Behaviour of the AI modules on controlled scenarios

Table 5.2 outlines eight Cashier service scenarios (4-minute average service time, capacity 10) alongside their recommendation engine outputs. These scenarios reflect the acceptance test suite cases, including empty, lopsided, tied, full, and closed queues, as well as existing ticket holders.

s.no	Waiting Q1 / Q2	CSP domain size	h(Q1), h(Q2) in min	Decision	Time saved (min)
1	0 / 0	2	0, 0	Queue 1 (tie, lowest index)	0
2	1 / 0	2	4, 0	Queue 2	4
3	1 / 1	2	4, 4	Queue 1 (tie, lowest index)	0
4	7 / 2	2	28, 8	Queue 2; imbalance 5	20
5	10 / 4	1 (Q1 full, pruned)	–, 16	Queue 2	24
6	10 / 10	0	–	No queue; all full message	0
7	Q1 closed / 5	1 (Q1 closed, pruned)	–, 20	Queue 2	0

Table 5.2 Recommendation engine output on controlled scenarios

In scenario 4 the engine moves a customer from a queue holding seven people to one holding two and saves 20 minutes; in scenario 5 the full queue is excluded by the CSP and the customer is saved 24 minutes relative to joining the saturated line. The tie-break is deterministic in scenarios 1 and 3, and the empty-domain and active-ticket cases return a clear refusal instead of an invalid assignment.

Table 5.3 shows the states derived by forward chaining for a queue of capacity ten. Note that the 60% boundary is inclusive, so six waiting customers already make a queue BUSY.

Waiting / status	Rule fired	Derived state	canJoin
0 / OPEN	R4	AVAILABLE	true
3 / OPEN	R4	AVAILABLE	true
6 / OPEN	R3	BUSY	true
9 / OPEN	R3	BUSY	true
10 / OPEN	R2	FULL	false
2 / CLOSED	R1	CLOSED	false

Table 5.3 Forward-chaining derivation (capacity = 10)

The backward-chaining prover returned a complete proof (G0 proved) for an open queue with three of ten places taken, and failed at sub-goal G1 (queue closed), G2 (queue full) and G4 (customer already has an active ticket) for the corresponding invalid requests. In the backtracking test, when the preferred Queue 1 was full (10/10) and Queue 2 held six, the module backtracked to Queue 2 with heuristic 24 minutes; when both queues were full it reported failure with the reason “All queues are currently at maximum capacity or closed.”

5.2.2 Acceptance test coverage

The automated acceptance suite seeds a fresh database, starts the server and drives the REST API with real HTTP calls. Its thirteen scenarios are listed in Table 5.4. They verify the AI decisions as well as the concurrency, FIFO, access-control and reporting guarantees.

No.	Behaviour verified
1	QR scan: first customer sees both queues empty, joins Queue 1 and receives token CASH-Q1-001; count becomes 1
2	Second customer sees 1 / 0, AI recommends Queue 2
3	Third customer sees a 1 / 1 tie; AI explains the tie-break and chooses Queue 1 (token CASH-Q1-002)
4	Staff call next (WAITING → IN_SERVICE, count decremented) and complete service (count unchanged)
5	Concurrency: at 9/10, two simultaneous joins – exactly one succeeds and the count never exceeds 10
6	Full queue and closed queue return the correct error messages
7	Cancel decrements by one, never below zero; a double cancel does not decrement twice
8	Single active ticket enforced; role-based access blocks unauthorised calls across roles and queues
9	Backtracking returns a fallback queue and trace when the preferred queue is full
10	Admin End Day resets counts to zero, creates a DailyReport, archives entries; the QR stays valid
11	Admin adds a new service with two queues which customers can use immediately
12	New staff registration selects a service and queue and receives an assignment
13	QR resolution returns all branch services and their live parallel queues

Table 5.4 Acceptance test scenarios

5.2.3 Policy simulation results

Table 5.5 reports the simulation results and Figures 5.1 and 5.2 show them graphically. At every load level the AI-balanced policy produced the lowest mean and 95th-percentile waits and the most even queues.

Load	Policy	Mean wait (min)	95th pct. wait (min)	Mean imbalance	Turned away
Light (ρ ≈ 0.50)	Always Queue 1	21.0	46.7	4.7	7.0%
	Random queue	4.6	17.8	0.9	0.0%
	AI-balanced	3.2	11.7	0.4	0.0%
Moderate (ρ ≈ 0.75)	Always Queue 1	32.2	58.2	8.0	30.5%
	Random queue	9.0	27.9	2.2	0.5%
	AI-balanced	6.5	19.6	0.6	0.0%
Heavy (ρ ≈ 0.95)	Always Queue 1	35.2	60.0	8.9	44.3%
	Random queue	16.7	43.7	3.5	4.7%
	AI-balanced	16.0	37.4	0.7	1.6%

Table 5.5 Simulation results (mean of 30 runs, 8-hour day)

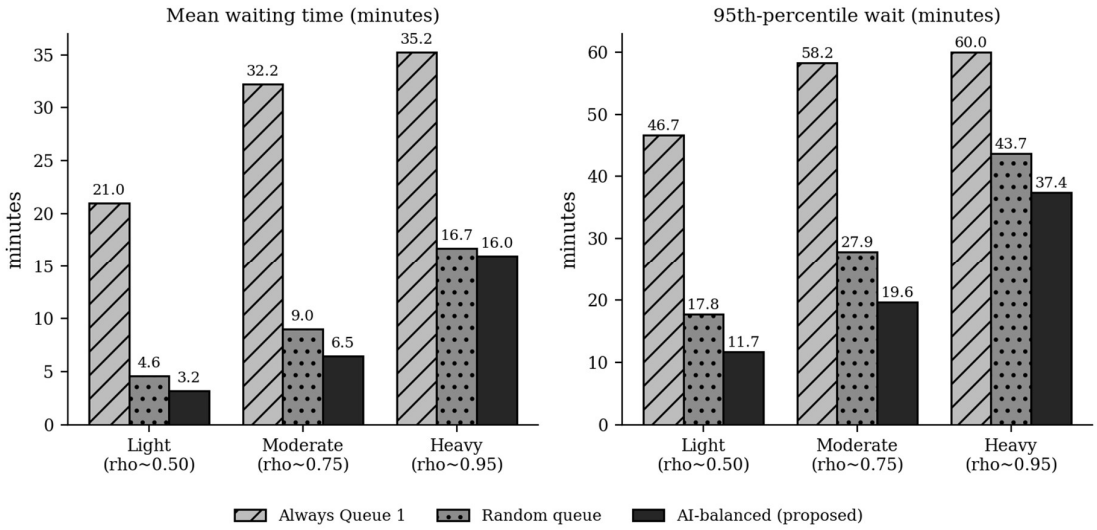


Figure 5.1 Mean and 95th-percentile waiting time by policy and load

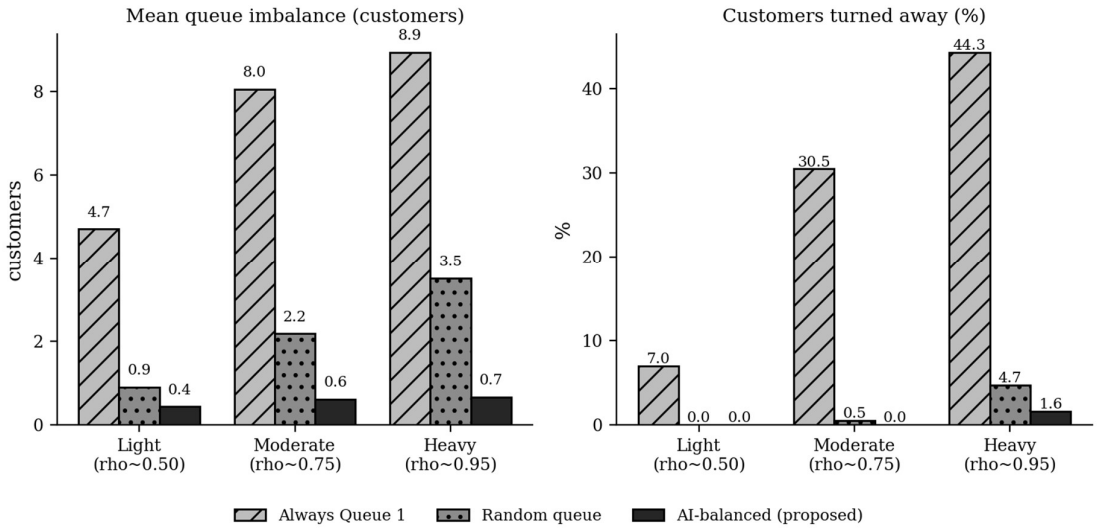


Figure 5.2 Queue imbalance and turned-away customers by policy and load

Compared with random choice, the AI policy lowered the mean wait by 29%, 29% and 5% at light, moderate and heavy load, and the 95th-percentile wait by 34%, 30% and 14%. The mean imbalance between the two queues fell by 52%, 72% and 81%. Under heavy load the proportion of customers turned away because a chosen queue was full dropped from 4.7% to 1.6%, because the AI policy only turns a customer away when every queue is full. The naive always-Queue-1 reference performs far worse, which illustrates the cost of ignoring queue state altogether.

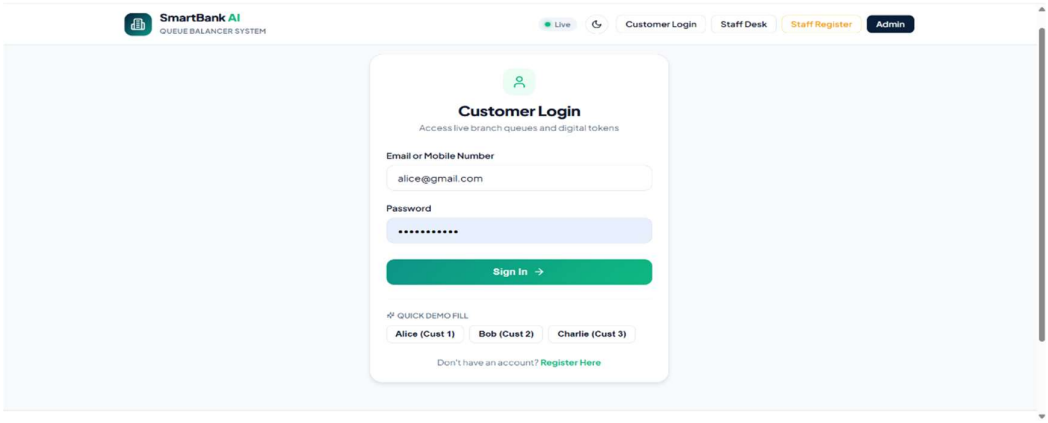
5.2.4 Discussion

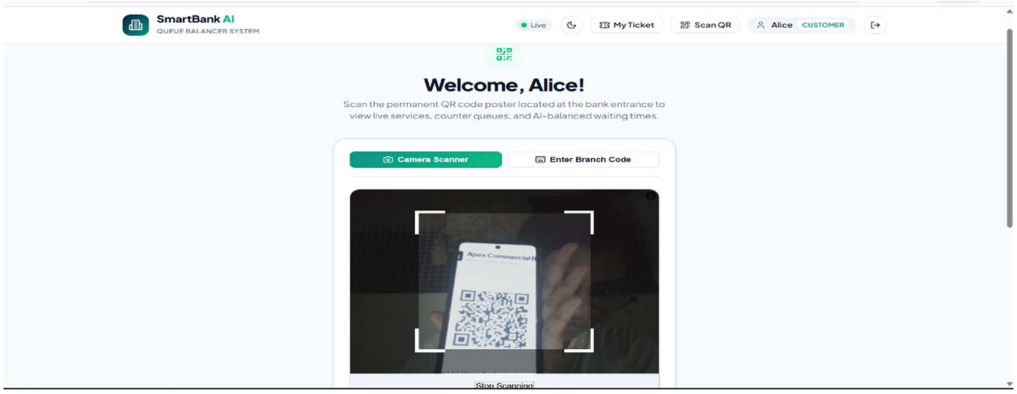
The simulation gives a consistent picture, but its limits should be understood. In this configuration both queues share one average service time, so the heuristic h(q) ranks queues exactly as the shortest-line rule does, and the performance gain over random choice is the known benefit of joining the shorter line (Winston, 1977). The advantage of the full system over a customer simply looking at two lines is that the decision is made on live, authoritative data, constraints are enforced so that a full or closed queue is never offered, the choice is explained, a stale choice is repaired by backtracking, and strict FIFO and concurrency safety are guaranteed. The relative gain is smaller under very heavy load because the counters themselves are close to saturation and no queue-selection policy can create service capacity.

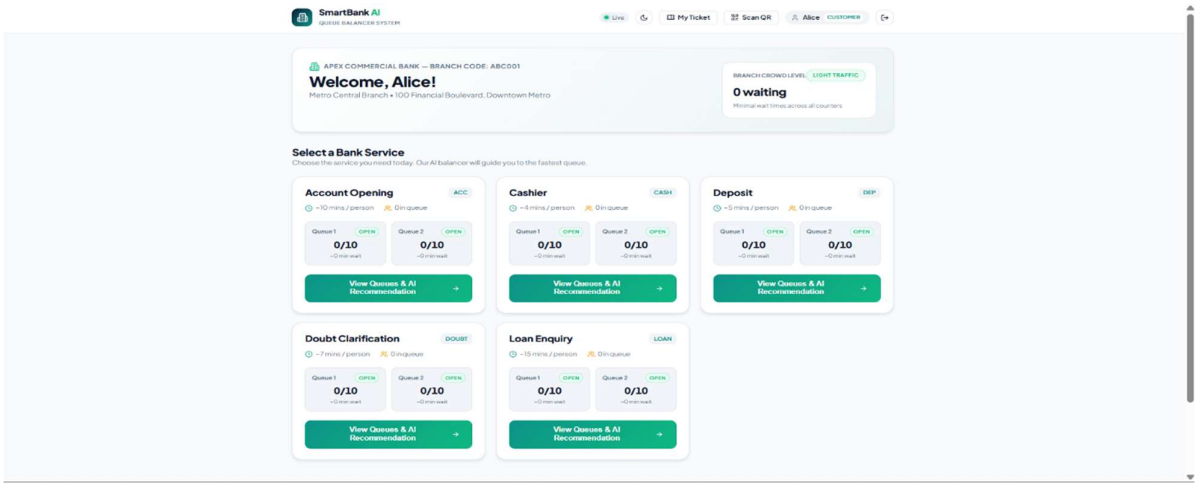
The results were obtained from a simplified model (one service, one counter per queue, Bernoulli arrivals) rather than from a live branch, so they indicate the direction and approximate size of the benefit and should not be read as a prediction for a specific bank. A pilot study with real arrival data is listed under future work.

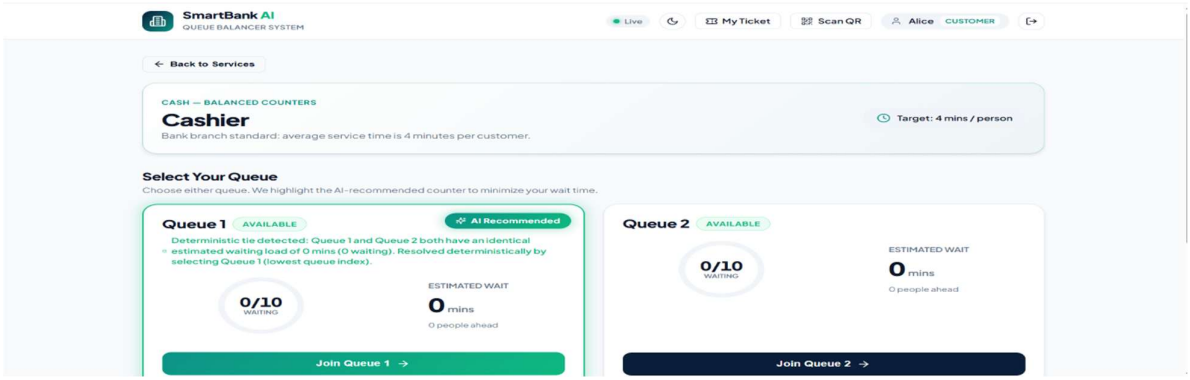
5.3 Output

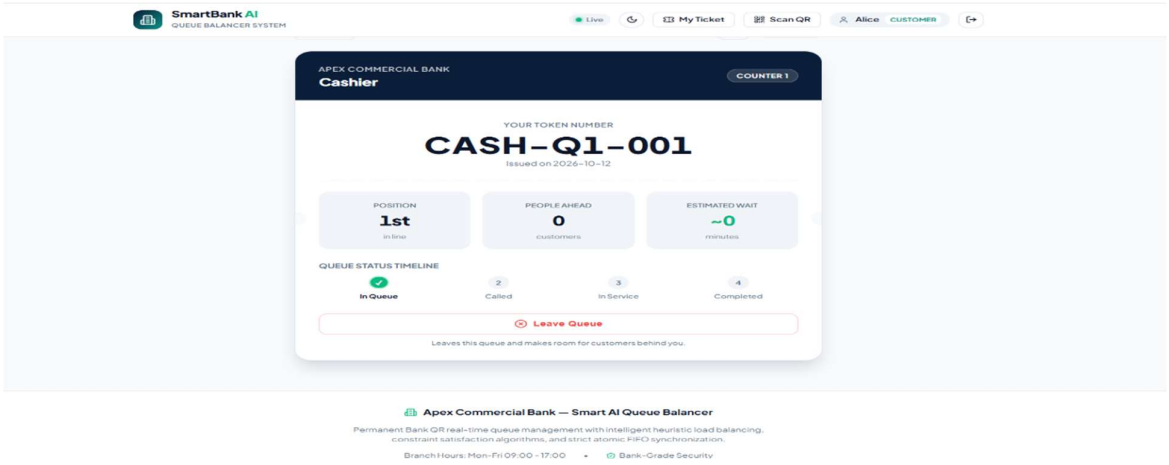
5.3.1 Customer:



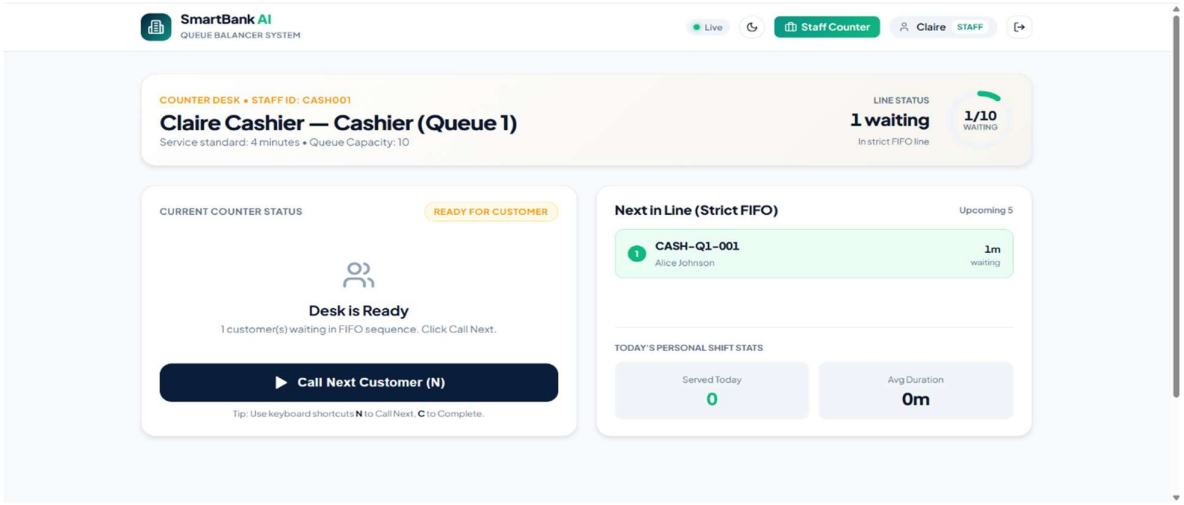


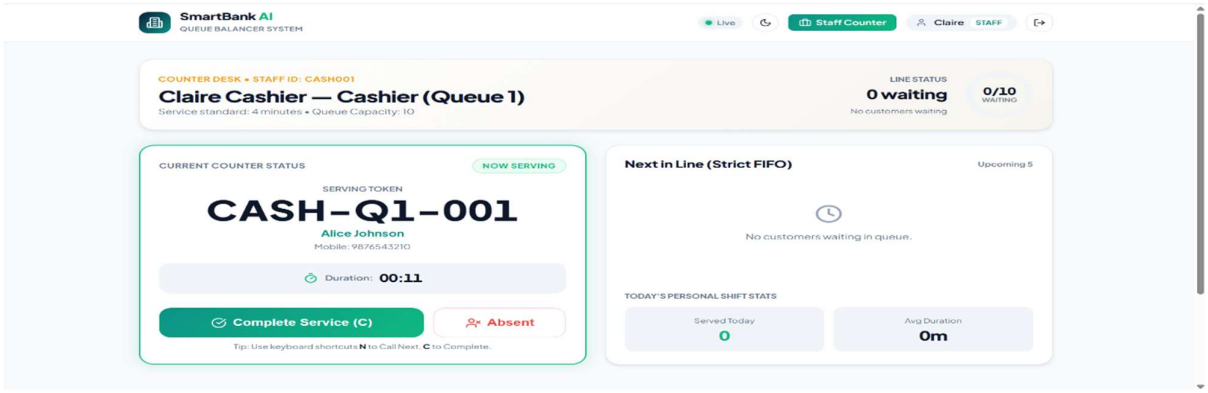



SmartBank AI
QUEUE BALANCER SYSTEM
Live
My Ticket
Scan QR
Alice CUSTOMER
Back to Services
CASH — BALANCED COUNTERS
Cashier
Bank branch standard: average service time is 4 minutes per customer.
Target: 4 mins / person
Select Your Queue
Choose either queue. We highlight the AI-recommended counter to minimize your wait time.
Queue 1 AVAILABLE
AI Recommended
Deterministic tie detected: Queue 1 and Queue 2 both have an identical estimated waiting load of 0 mins (0 waiting). Resolved deterministically by selecting Queue 1 (lowest queue index).
0/10
WAITING
ESTIMATED WAIT
0 mins
0 people ahead
Join Queue 1
Queue 2 AVAILABLE
0/10
WAITING
ESTIMATED WAIT
0 mins
0 people ahead
Join Queue 2

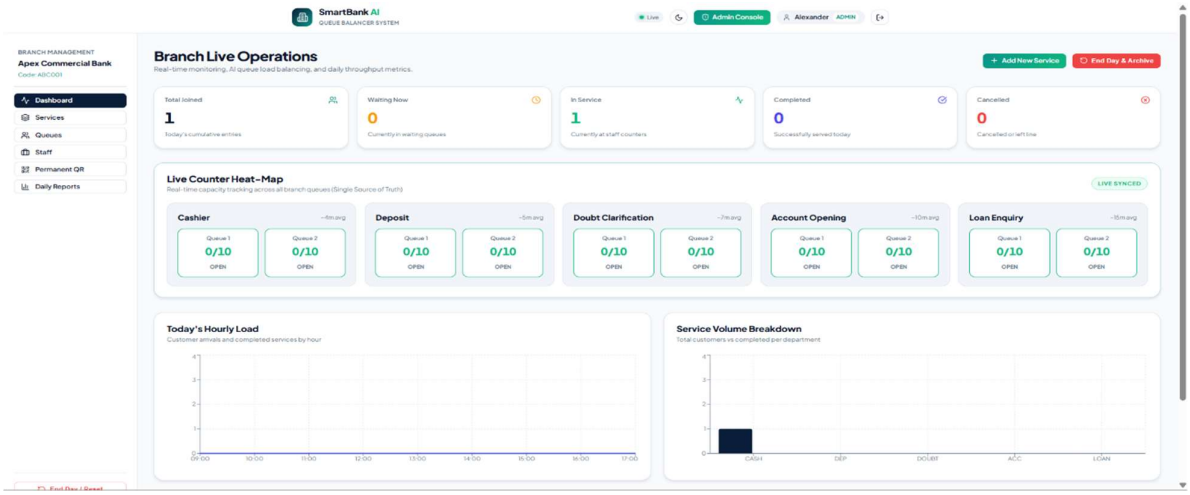

SmartBank AI
QUEUE BALANCER SYSTEM
Live
My Ticket
Scan QR
Alice CUSTOMER
APEX COMMERCIAL BANK
Cashier
COUNTER 1
YOUR TOKEN NUMBER
CASH-Q1-001
Issued on 2026-10-12
POSITION
1st
in line
PEOPLE AHEAD
0
customers
ESTIMATED WAIT
~0
minutes
QUEUE STATUS TIMELINE
In Queue
Called
In Service
Completed
Leave Queue
Leaves this queue and makes room for customers behind you.
Apex Commercial Bank — Smart AI Queue Balancer
Permanent Bank QR real-time queue management with intelligent heuristic load balancing, constraint satisfaction algorithms, and strict atomic FIFO synchronization.
Branch Hours: Mon-Fri 09:00 - 17:00
Bank-Grade Security

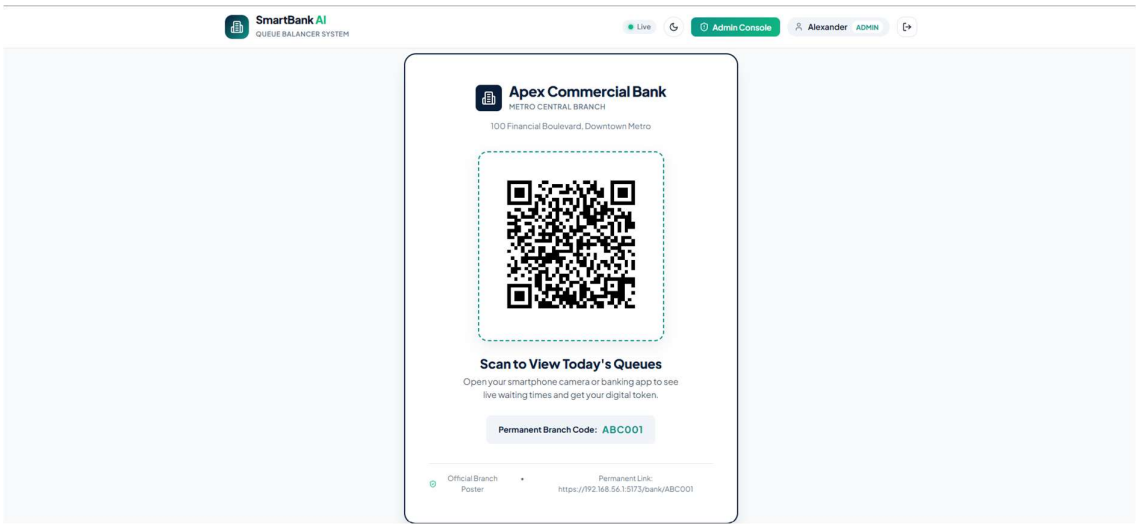
5.3.2 Staff:


SmartBank AI
QUEUE BALANCER SYSTEM
Live
Staff Counter
Claire STAFF
COUNTER DESK • STAFF ID: CASH001
Claire Cashier — Cashier (Queue 1)
Service standard: 4 minutes • Queue Capacity: 10
LINE STATUS
1 waiting
In strict FIFO line
1/10
WAITING
CURRENT COUNTER STATUS
READY FOR CUSTOMER
Desk is Ready
1 customer(s) waiting in FIFO sequence. Click Call Next.
Call Next Customer (N)
Tip: Use keyboard shortcuts N to Call Next, C to Complete.
Next in Line (Strict FIFO)
Upcoming 5
1
CASH-Q1-001
Alice Johnson
1m
waiting
TODAY'S PERSONAL SHIFT STATS
Served Today
0
Avg Duration
0m


SmartBank AI
QUEUE BALANCER SYSTEM
Live
Staff Counter
Claire STAFF
COUNTER DESK • STAFF ID: CASH001
Claire Cashier — Cashier (Queue 1)
Service standard: 4 minutes • Queue Capacity: 10
LINE STATUS
0 waiting
No customers waiting
0/10
WAITING
CURRENT COUNTER STATUS
NOW SERVING
SERVING TOKEN
CASH-Q1-001
Alice Johnson
Mobile: 9876543210
Duration: 00:11
Complete Service (C)
Absent
Tip: Use keyboard shortcuts N to Call Next, C to Complete.
Next in Line (Strict FIFO)
Upcoming 5
No customers waiting in queue.
TODAY'S PERSONAL SHIFT STATS
Served Today
0
Avg Duration
0m

5.3.3 Admin:


SmartBank AI
QUEUE BALANCER SYSTEM
Admin Console
Alexander ADMIN
BRANCH MANAGEMENT
Apex Commercial Bank
Code: ABC001
Dashboard
Services
Queues
Staff
Permanent QR
Daily Reports
Branch Live Operations
Real-time monitoring, AI queue load balancing, and daily throughput metrics.
+ Add New Service
End Day & Archive
Total Joined
1
Waiting Now
0
In Service
1
Completed
0
Cancelled
0
Live Counter Heat-Map
Real-time capacity tracking across all branch queues (Single Source of Truth)
LIVE SYNCED
Cashier
Deposit
Doubt Clarification
Account Opening
Loan Enquiry
0/10
OPEN
Today's Hourly Load
Customer arrivals and completed services by hour
Service Volume Breakdown
Total customers vs completed per department


SmartBank AI
QUEUE BALANCER SYSTEM
Live
Admin Console
Alexander ADMIN
Apex Commercial Bank
METRO CENTRAL BRANCH
100 Financial Boulevard, Downtown Metro
Scan to View Today's Queues
Open your smartphone camera or banking app to see live waiting times and get your digital token.
Permanent Branch Code: ABC001
Official Branch Poster
Permanent Link:
https://192.168.56.1:5173/bank/ABC001

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project designed and implemented an AI-based smart queue balancer for a bank branch. A permanent QR code gives any customer instant, install-free access to live queues. A modular AI engine combining a rational-agent recommender, a heuristic function, greedy best-first selection, constraint satisfaction, backtracking, forward chaining and backward chaining selects a valid, low-wait queue and explains its reasoning. Atomic database operations, strict FIFO ordering, role-based security and Socket.IO synchronisation keep the shared state correct and every screen current.

Scenario runs showed that the engine prunes invalid queues, resolves ties deterministically, backtracks when a race is lost and proves eligibility with an inspectable trace. In simulation, the engine reduced mean queue imbalance by roughly half to four-fifths and shortened the 95th-percentile wait by 14–34% relative to random queue choice. The project therefore demonstrates that techniques from the Foundations of Artificial Intelligence course can be integrated into a practical, real-time service application.

Future work

- Learn service times per counter or per transaction type from history so that the heuristic differentiates queues and staff of different speed.
- Add a predictive model for arrival rates to open or close counters proactively and to give more accurate wait estimates.
- Replace the greedy selection with a multi-step search or optimisation that balances several services and priority customers (senior citizens, accessibility needs) at once.
- Provide notifications through SMS, WhatsApp or push so customers can leave the branch and return when their turn is near.
- Support multiple branches with a central analytics dashboard, and integrate with core-banking appointment data.
- Run a pilot at a real branch to validate the model against measured arrival and service data and to collect customer satisfaction feedback.

CHAPTER 7

REFERENCES

- Brachman, R. J. and Levesque, H. J. (2004) *Knowledge Representation and Reasoning*, Morgan Kaufmann, San Francisco.
- Dechter, R. (2003) *Constraint Processing*, Morgan Kaufmann, San Francisco.
- Golomb, S. W. and Baumert, L. D. (1965) 'Backtrack programming', *Journal of the ACM*, Vol.12, No.4, pp.516–524.
- Gross, D. and Harris, C. M. (1998) *Fundamentals of Queueing Theory*, 3rd edn, John Wiley & Sons, New York.
- Haight, F. A. (1957) 'Queueing with balking', *Biometrika*, Vol.44, No.3/4, pp.360–369.
- Hart, P. E., Nilsson, N. J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, Vol.4, No.2, pp.100–107.
- Jones, M., Bradley, J. and Sakimura, N. (2015) 'JSON Web Token (JWT)', *RFC 7519*, Internet Engineering Task Force.
- Kleinrock, L. (1975) *Queueing Systems, Volume 1: Theory*, John Wiley & Sons, New York.
- Larson, R. C. (1987) 'Perspectives on queues: social justice and the psychology of queueing', *Operations Research*, Vol.35, No.6, pp.895–905.
- Little, J. D. C. (1961) 'A proof for the queuing formula: $L = \lambda W$', *Operations Research*, Vol.9, No.3, pp.383–387.
- Mackworth, A. K. (1977) 'Consistency in networks of relations', *Artificial Intelligence*, Vol.8, No.1, pp.99–118.
- Pearl, J. (1984) *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, Reading, MA.
- Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*, 4th edn, Pearson, Hoboken, NJ.
- Winston, W. (1977) 'Optimality of the shortest line discipline', *Journal of Applied Probability*, Vol.14, No.1, pp.181–189.